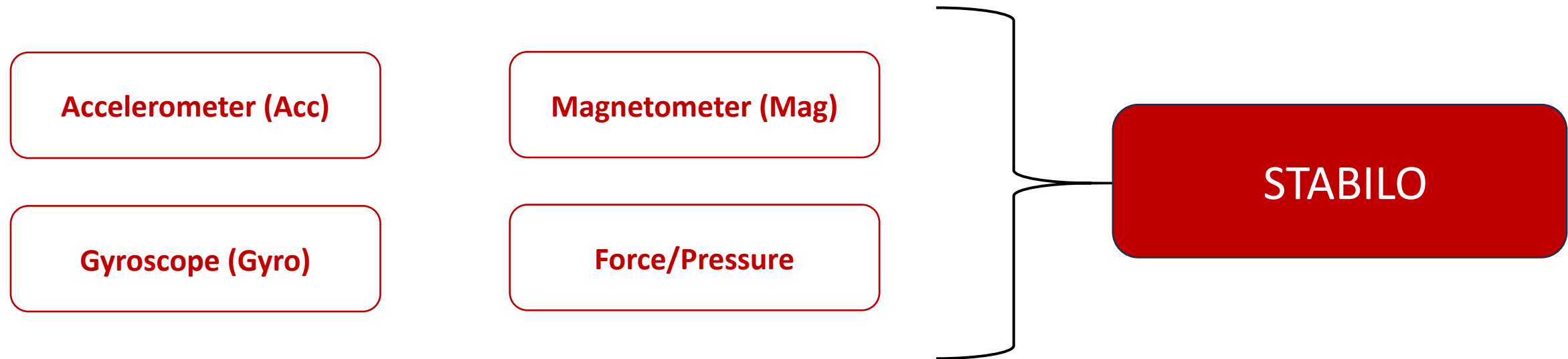


Writer ID Prediction using BiLSTM

Sayed Fatema Tuj Zohura, Sumon Kazi, Symon Islam,
Vishwa Balasubramanian

Identify *which person* wrote a handwriting sample (**writer-ID classification**) using sensor data from an IMU-enhanced pen.



A standard **LSTM** only uses **past context**

BiLSTM uses **past and future context**

BiLSTM captures these **global motion characteristics** better

BiLSTM is a **natural and stronger choice writer ID classification**

We experimented the baseline model **BILSTM**



BiLSTM (Bidirectional LSTM) is simply **two LSTMs together**:

1. A forward LSTM: $x_1 \rightarrow \dots \rightarrow x_T$
2. A backward LSTM: $x_T \rightarrow \dots \rightarrow x_1$

Representations are **combined**, means a BiLSTM uses **both past and future context**

We used Two different datasets

1. **OnHW-words500_dep** dataset (writer-dependent)
2. **OnHW-equations_dep** dataset (writer-dependent)

OnHW-words500_dep dataset (writer-dependent)

For Writer Identification we needed a dataset with the label of writer's ID or name. In this dataset, there is the label with writer's IDs, who wrote each recording.

```
def load_pkl(path):
    with open(path, "rb") as f:
        return pickle.load(f)

X_train = load_pkl(os.path.join(base_path, "all_x_dat_train_imu.pkl"))
y_train = load_pkl(os.path.join(base_path, "train_ids.pkl"))

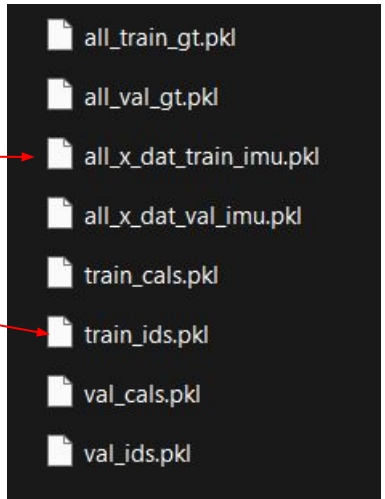
X_val = load_pkl(os.path.join(base_path, "all_x_dat_val_imu.pkl"))
y_val = load_pkl(os.path.join(base_path, "val_ids.pkl"))

print("Train samples:", len(X_train))
print("Val samples:", len(X_val))

print("Example time-series shape:", X_train[0].shape)
print("Unique writer IDs:", np.unique(y_train))
```

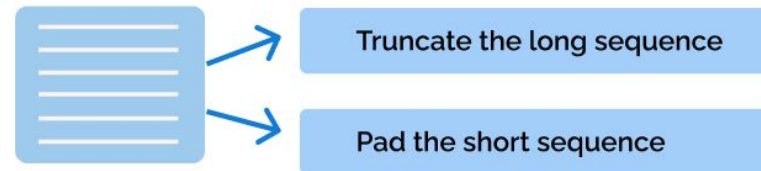
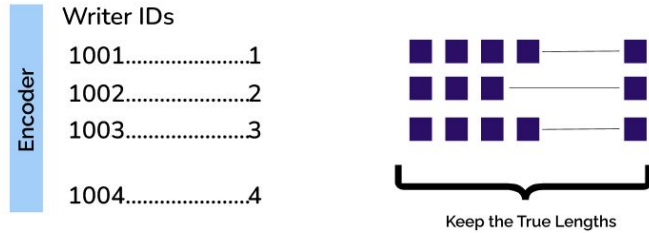
Train samples: 20176
Val samples: 5042
Example time-series shape: (220, 13)
Unique writer IDs: [0 1 2 3 5 6 7 8 9 10 11 12 13 14
15 16 17 18 1001 1002 1003 1004 1005 1006 1007 1008 1009 1011
1050 1051 1053 1054 1060 1061 1071 1072 1073 1080 1086 1090 2001 2002
2003 2004 2005 2006 2007 4001 4002 4005 4006 4008 4009]

Fold 0



- all_train_gt.pkl
- all_val_gt.pkl
- all_x_dat_train_imu.pkl
- all_x_dat_val_imu.pkl
- train_cals.pkl
- train_ids.pkl
- val_cals.pkl
- val_ids.pkl

Preprocessing



```
print("X_train example shape:", X_train[0].shape)
print("X_val example shape:", X_val[0].shape)
```

X_train example shape: (220, 13)
X_val example shape: (206, 13)

time step	feature 1	feature 2	...	feature 13
0	value	value	...	value
1	value	value	...	value
...	...	...	...	...
219	value	value	...	value

time step	feature 1	feature 2	...	feature 13
0	value	value	...	value
1	value	value	...	value
...	...	...	...	...
204	value	value	...	value
205	value	value	...	value

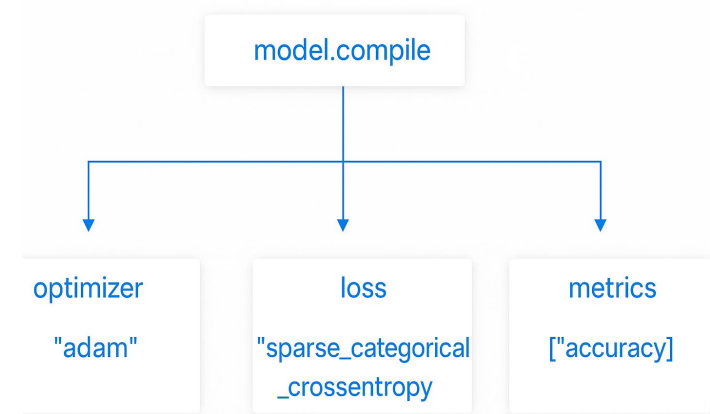
Train padded shape: (20176, 100, 13)
Val padded shape: (5042, 100, 13)

time step	feature 1	feature 2	...	feature 13
0	value	value	...	value
1	value	value	...	value
...	...	...	...	...
98	value	value	...	value
99	value	value	...	value

Model BiLSTM Dataset 1

We experimented the baseline model **BILSTM**

Layer (type)	Output Shape	Param #
bidirectional (Bidirectional)	(None, 100, 128)	39,936
dropout (Dropout)	(None, 100, 128)	0
bidirectional_1 (Bidirectional)	(None, 64)	41,216
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout_2 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 53)	3,445



Result Dataset 1

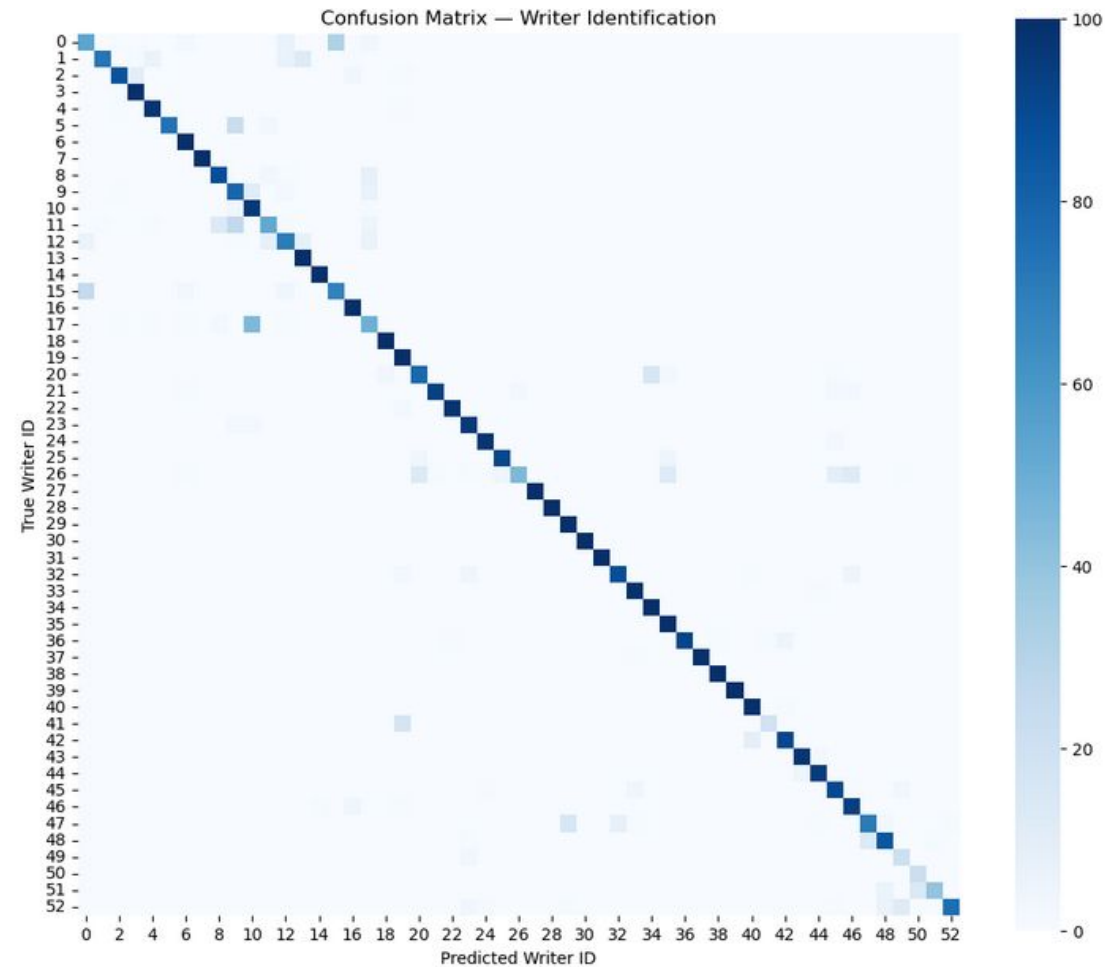
Prediction

1/1 ————— 1s 1s/step
Sample 3894: Pred = 38, True = 38
1/1 ————— 0s 105ms/step
Sample 594: Pred = 5, True = 5
1/1 ————— 0s 110ms/step
Sample 2441: Pred = 24, True = 24
1/1 ————— 0s 121ms/step
Sample 310: Pred = 3, True = 3
1/1 ————— 0s 120ms/step
Sample 1623: Pred = 16, True = 16

Evaluation

158/158 ————— 5s 29ms/step
Final Validation Accuracy: 0.8857596191987307

Confusion Matrix Dataset 1



We used the **OnHW-equations_dep** dataset (writer-dependent).

In this we have **Writers Id** which is used to train the model to identify which sample is written by whom

For **writer-ID classification**, this is the correct choice, recognize writers it has seen during training.

If a writer never appears in training, you **cannot** predict that writer ID at test time

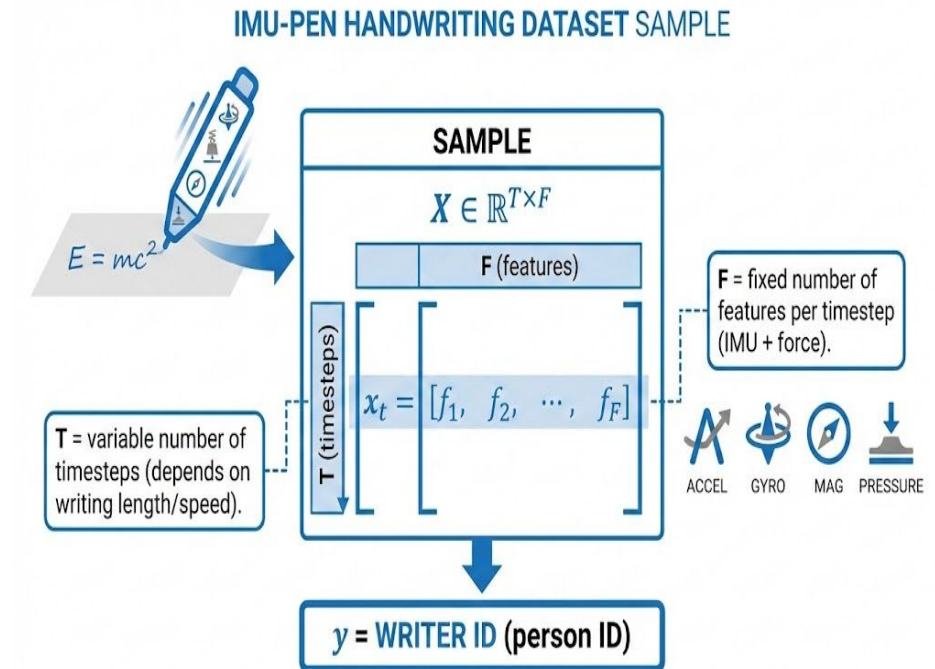
What one Sample Looks Like

One sample = one complete equation written by one person

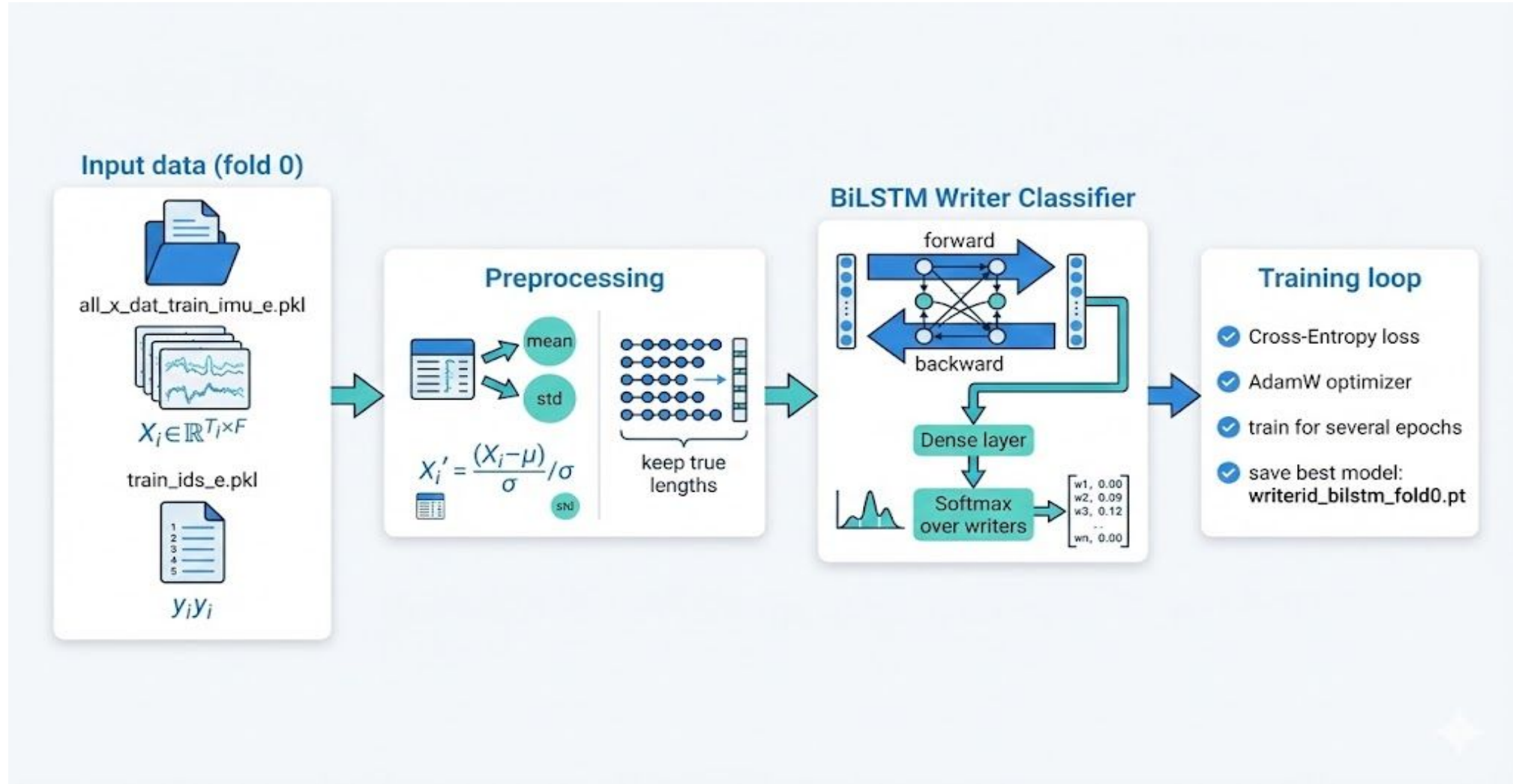
Input X : a multivariate time series with number of time steps and number of time features

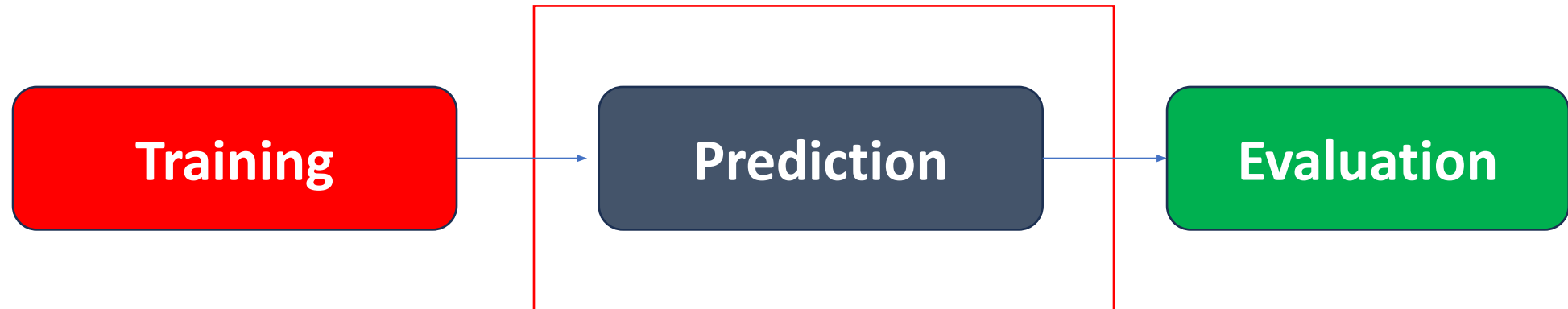
Label y : the writer ID (person ID)

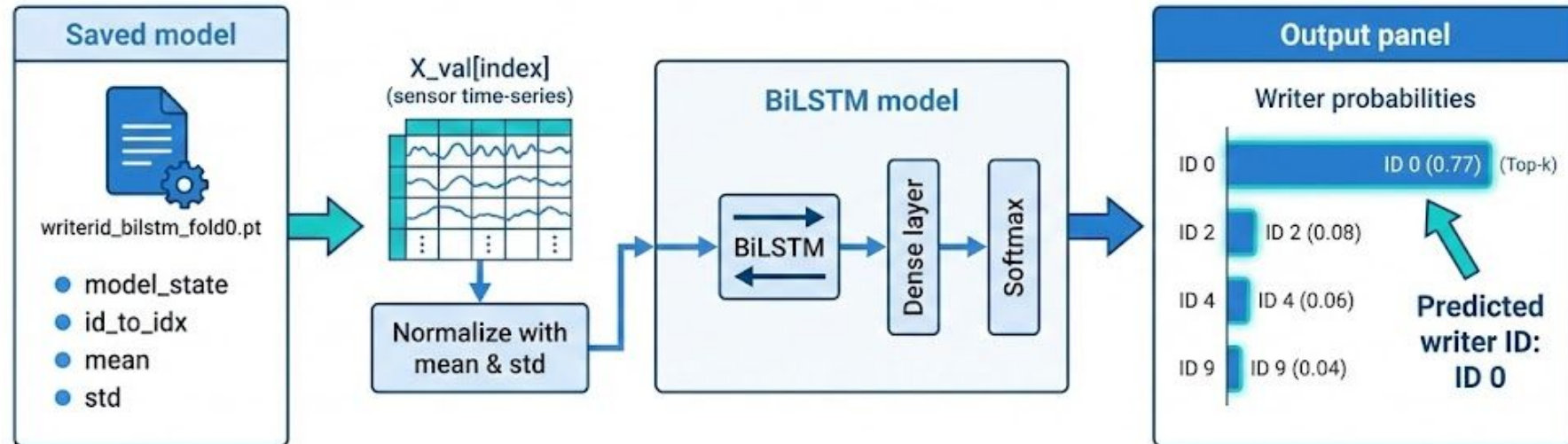
So each Training sample is the pair (X, y) : Sequence of pen sensor readings and the writer's ID



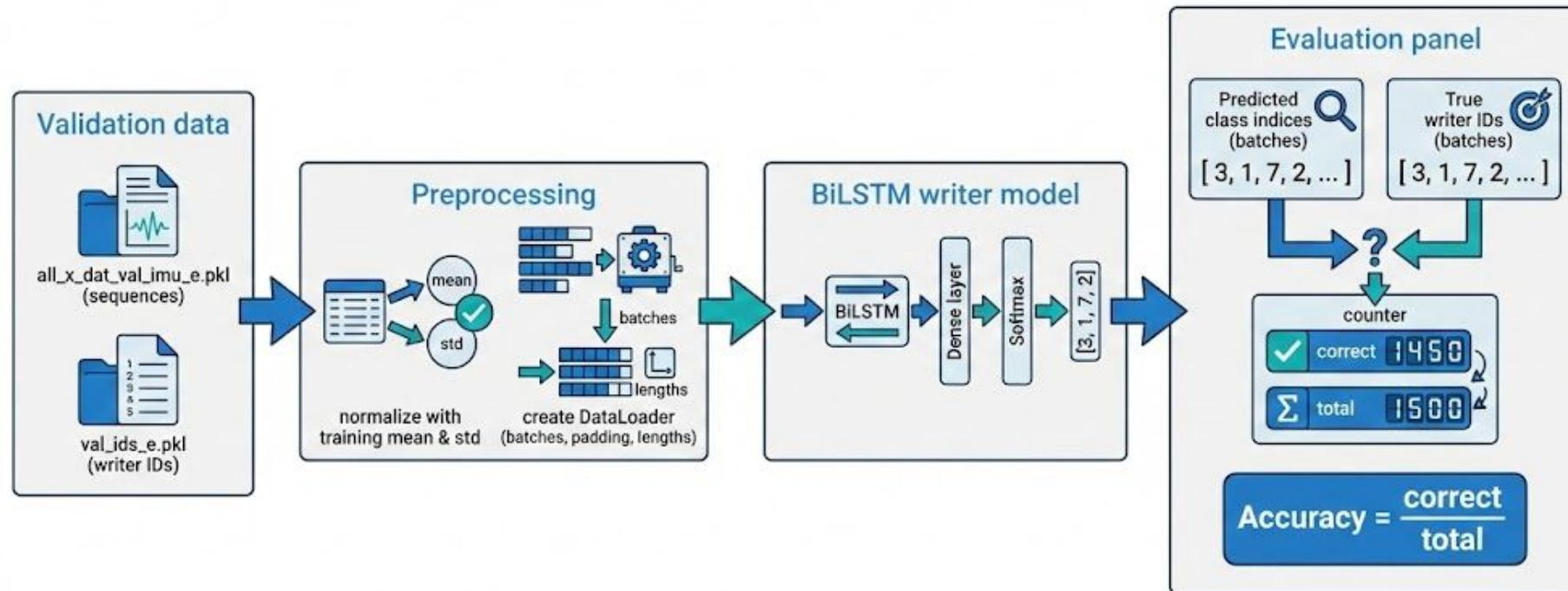












Training

```
> .venv
> onhw-chars_2021-06-30
> OnHW-equations_dep
> OnHW-symbols_equations_indep
+ evaluvate.py
+ main.py
+ predict_writer.py
+ writerid_bilstm_pt
```

Prediction

```
• (.venv) PS C:\Users\bvish\Downloads\lstm_handwriting_project> python predict_writer.py --index 999
Split=val fold=0 index=999 (requested 999)
True writer ID: 1027
Pred writer ID: 1027 (confidence=0.833)

Top predictions:
1027 0.833
1026 0.116
1008 0.012
1020 0.007
1055 0.005
```

Evaluation

```
• (.venv) PS C:\Users\bvish\Downloads\lstm_handwriting_project> python evaluvate.py
Fold0 VAL accuracy: 93.92% (1991/2120)
```

BiLSTM	Accuracy
Dataset 1 Words 500	88 %
Dataset 2 equations	93.92 %

Advanced model: Neural ODEs

Why it might help :

1. **Smooth, Continuous handwriting** dynamics
2. Handle **irregular sampling**
3. Potentially more **compact and interpretable**

Plan:

Build a Neural ODE-based model

Compare performance Neural ODE vs BiLSTM vs LSTM